Just Random Forest

June Lemieux

2024-12-14

libraries/etc

```r
library(tidymodels)
```

```
## -- Attaching packages -------------------------------------- tidymodels 1.2.0 --

## v broom        1.0.7     v recipes      1.1.0
## v dials        1.3.0     v rsample      1.2.1
## v dplyr        1.1.4     v tibble       3.2.1
## v ggplot2      3.5.1     v tidyr        1.3.1
## v infer        1.0.7     v tune         1.2.1
## v modeldata    1.4.0     v workflows    1.1.4
## v parsnip      1.2.1     v workflowsets 1.1.0
## v purrr        1.0.2     v yardstick    1.3.1

## -- Conflicts ----------------------------------------- tidymodels_conflicts() --
## x purrr::discard() masks scales::discard()
## x dplyr::filter()  masks stats::filter()
## x dplyr::lag()     masks stats::lag()
## x recipes::step()  masks stats::step()
## * Use tidymodels_prefer() to resolve common conflicts.
```

```r
library(ggplot2)
library(vip)
```

```
## Warning: package 'vip' was built under R version 4.4.2

##
## Attaching package: 'vip'

## The following object is masked from 'package:utils':
##
##     vi
```

```r
library(ROCR)
```

```
## Warning: package 'ROCR' was built under R version 4.4.2
```

```r
options(scipen=4)
```

```r
## Data Proocessing
# Initial data import
customer_data <- read.csv("~/00 Merrimack/DSE6111/Course Project/customer_data_1.csv", header = TRUE)

customer_data$Attrition_Flag <- ifelse(customer_data$Attrition_Flag == "Attrited Customer", 1, 0)
customer_data$Attrition_Flag <- factor(customer_data$Attrition_Flag)

# check levels
levels(customer_data$Attrition_Flag)
```

```
## [1] "0" "1"
```

```r
# set seed
set.seed(10)

# get rid of ClientNum identifier
customer_data <- customer_data[ -c(1) ]

# create data split
data_split <- initial_split(customer_data, strata = "Attrition_Flag", prop = 0.75)

# create training and test sets
training_set <- training(data_split)
test_set  <- testing(data_split)

# row count per set
nrow(training_set)
```

```
## [1] 7595
```

```r
nrow(test_set)
```

```
## [1] 2532
```

```r
# use cv folds and tune
cv_set <- vfold_cv(training_set, v = 10)
```

```r
attrition_recipe_rf <-
  recipe(
    Attrition_Flag ~ Months_on_book + Total_Relationship_Count + Months_Inactive_12_mon + Contacts_Coun
      Total_Amt_Chng_Q4_Q1 + Total_Trans_Amt + Total_Trans_Ct + Total_Ct_Chng_Q4_Q1 +
      Avg_Utilization_Ratio,
    data = training_set
  ) %>%
  step_dummy(all_nominal_predictors()) %>%
  step_center(all_predictors()) %>%
  step_scale(all_predictors())

rf_model <-
```

```r
  rand_forest(mtry = tune()) %>%
  set_engine("ranger", importance = "impurity") %>%
  set_mode("classification")

rf_workflow <-
  workflow() %>%
  add_recipe(attrition_recipe_rf) %>%
  add_model(rf_model)
```

```r
################################# use tune_grid to find best mtry ##############################

rf_mtry_grid <- tibble(mtry = 10^seq(-1, 0, length.out = 15))

rf_results <-
  rf_workflow %>%
  tune_grid(cv_set,
            grid = rf_mtry_grid,
            control = control_grid(save_pred = TRUE),
            metrics = metric_set(roc_auc, accuracy))
```

```
## Warning: package 'ranger' was built under R version 4.4.2
```

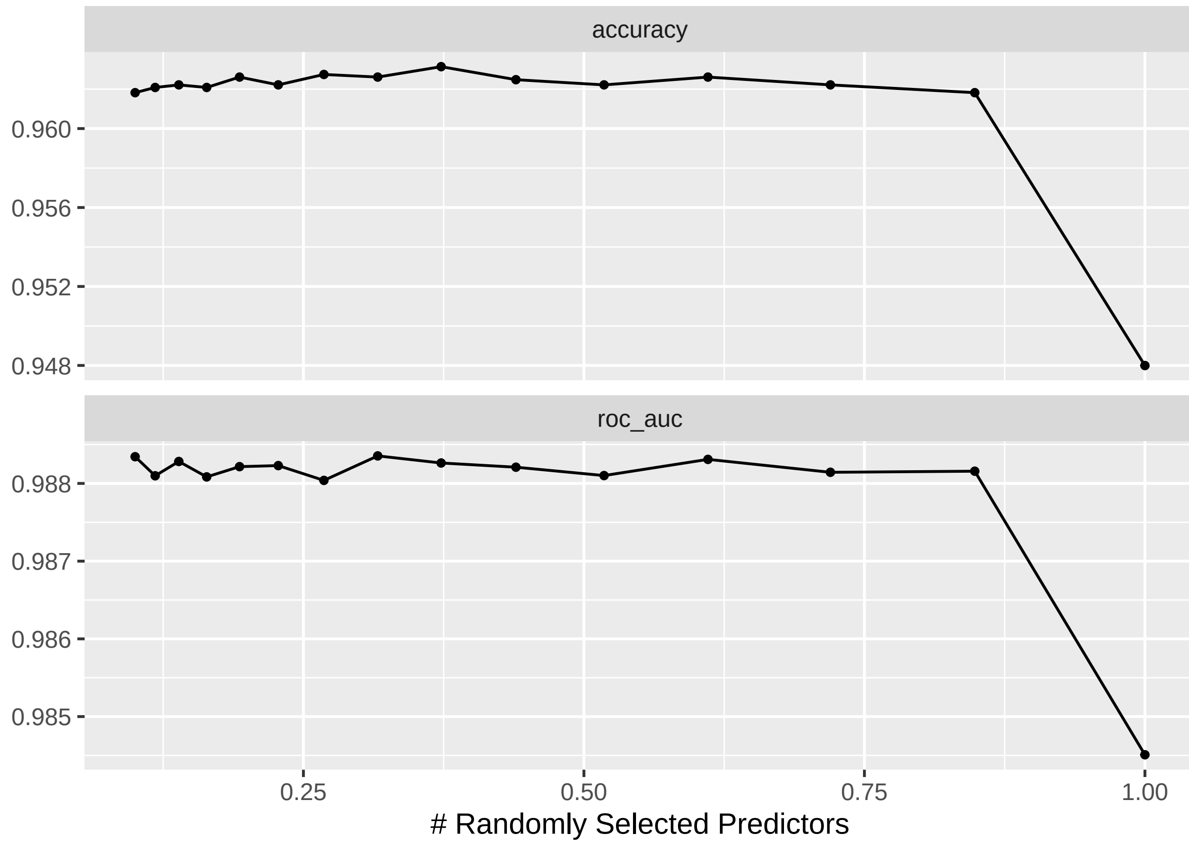
```r
rf_rs_metrics <- rf_results %>% collect_metrics()
rf_rs_predictions <- rf_results %>% collect_predictions()
rf_rs_metrics
```

```
## # A tibble: 30 x 7
##     mtry .metric  .estimator  mean     n std_err .config
##    <dbl> <chr>    <chr>      <dbl> <int>   <dbl> <chr>
##  1 0.1   accuracy binary     0.962    10 0.00194 Preprocessor1_Model01
##  2 0.1   roc_auc  binary     0.988    10 0.00117 Preprocessor1_Model01
##  3 0.118 accuracy binary     0.962    10 0.00196 Preprocessor1_Model02
##  4 0.118 roc_auc  binary     0.988    10 0.00121 Preprocessor1_Model02
##  5 0.139 accuracy binary     0.962    10 0.00193 Preprocessor1_Model03
##  6 0.139 roc_auc  binary     0.988    10 0.00119 Preprocessor1_Model03
##  7 0.164 accuracy binary     0.962    10 0.00205 Preprocessor1_Model04
##  8 0.164 roc_auc  binary     0.988    10 0.00117 Preprocessor1_Model04
##  9 0.193 accuracy binary     0.963    10 0.00201 Preprocessor1_Model05
## 10 0.193 roc_auc  binary     0.988    10 0.00117 Preprocessor1_Model05
## # i 20 more rows
```

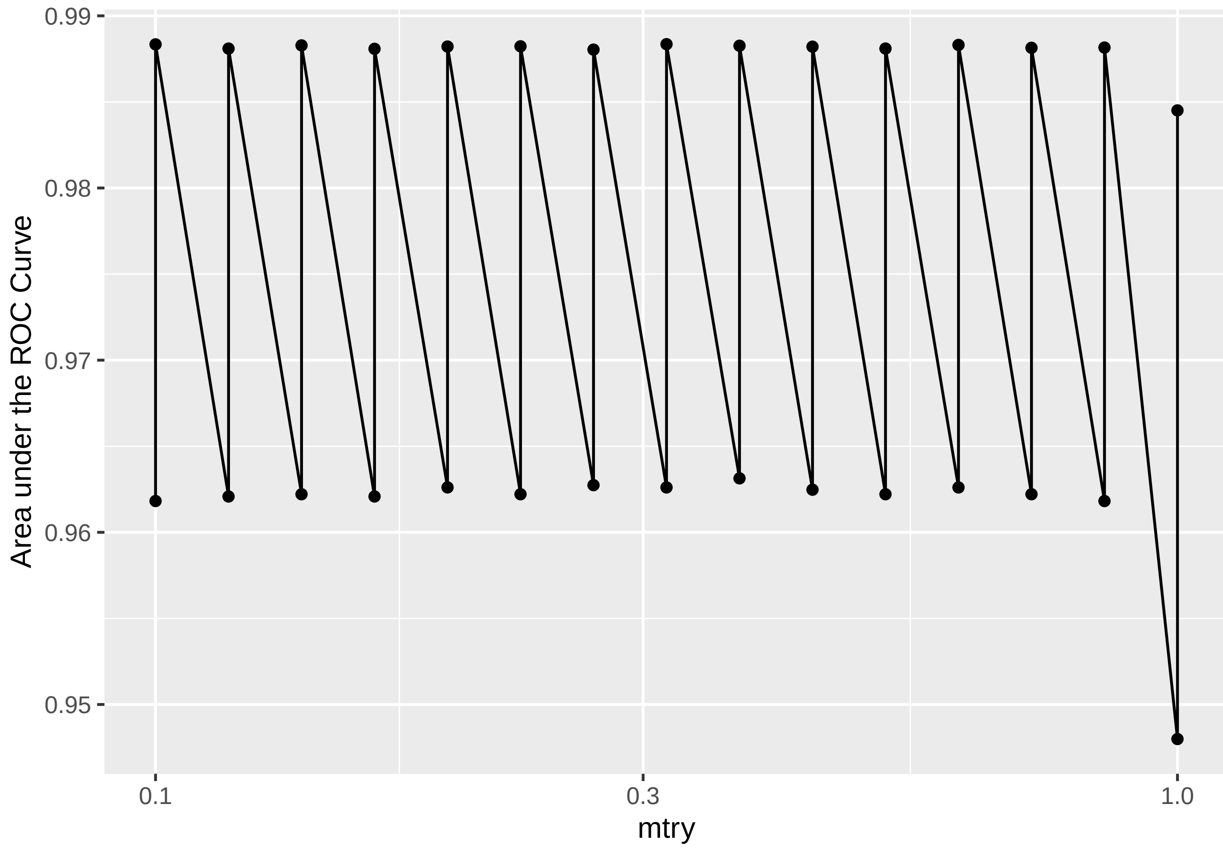
```r
# roc_auc and accuracy
rf_results %>% show_best(metric = "roc_auc")
```

```
## # A tibble: 5 x 7
##    mtry .metric .estimator  mean     n std_err .config
##   <dbl> <chr>   <chr>      <dbl> <int>   <dbl> <chr>
## 1 0.316 roc_auc binary     0.988    10 0.00117 Preprocessor1_Model08
## 2 0.1   roc_auc binary     0.988    10 0.00117 Preprocessor1_Model01
## 3 0.611 roc_auc binary     0.988    10 0.00116 Preprocessor1_Model12
## 4 0.139 roc_auc binary     0.988    10 0.00119 Preprocessor1_Model03
## 5 0.373 roc_auc binary     0.988    10 0.00115 Preprocessor1_Model09
```

```r
autoplot(rf_results)
```

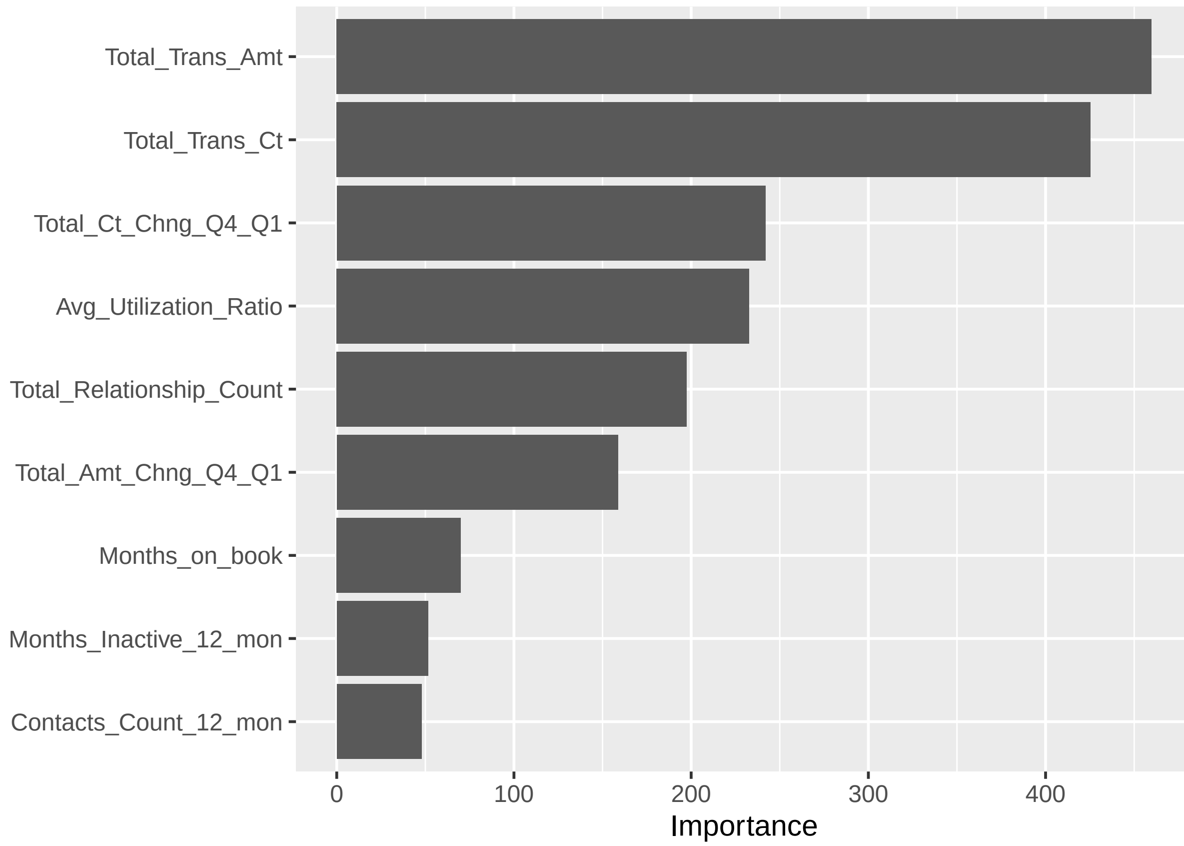



```r
best_auc <- rf_results %>% select_best(metric = "roc_auc")
best_acc <- rf_results %>% select_best(metric = "accuracy")
best_neighbors <- rf_results %>% select_best(metric = 'roc_auc')


rf_plot <-
  rf_results %>%
  collect_metrics() %>%
  ggplot(aes(x = mtry, y = mean)) +
  geom_point() +
  geom_line() +
  ylab("Area under the ROC Curve") +
  scale_x_log10(labels = scales::label_number())

rf_plot
```




```
final_rf_workflow <- finalize_workflow(
  rf_workflow,
  best_auc
)

last_rf_fit <-
  final_rf_workflow %>%
  last_fit(data_split)

rf_metrics <- last_rf_fit %>% collect_metrics()
```

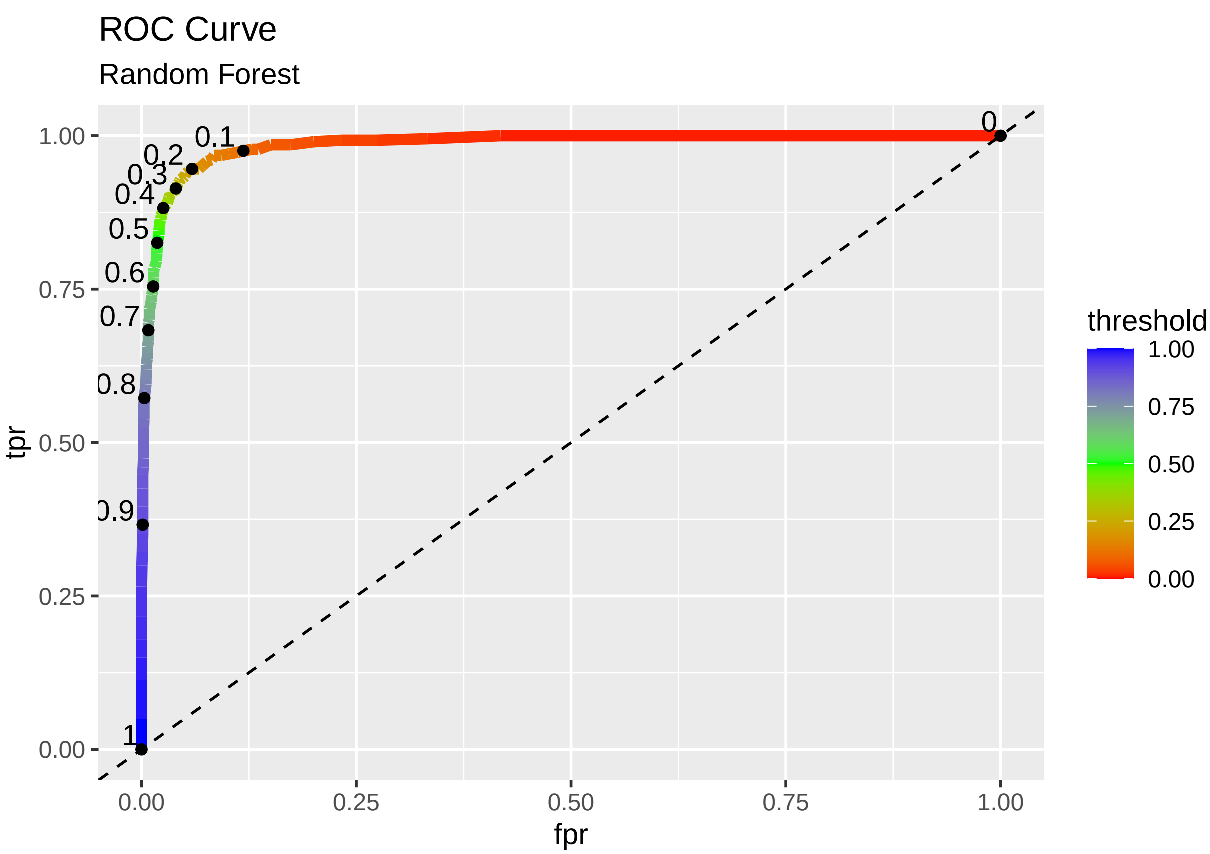
```
last_rf_fit %>%
  extract_fit_parsnip() %>%
  vip(num_features = 10)
```



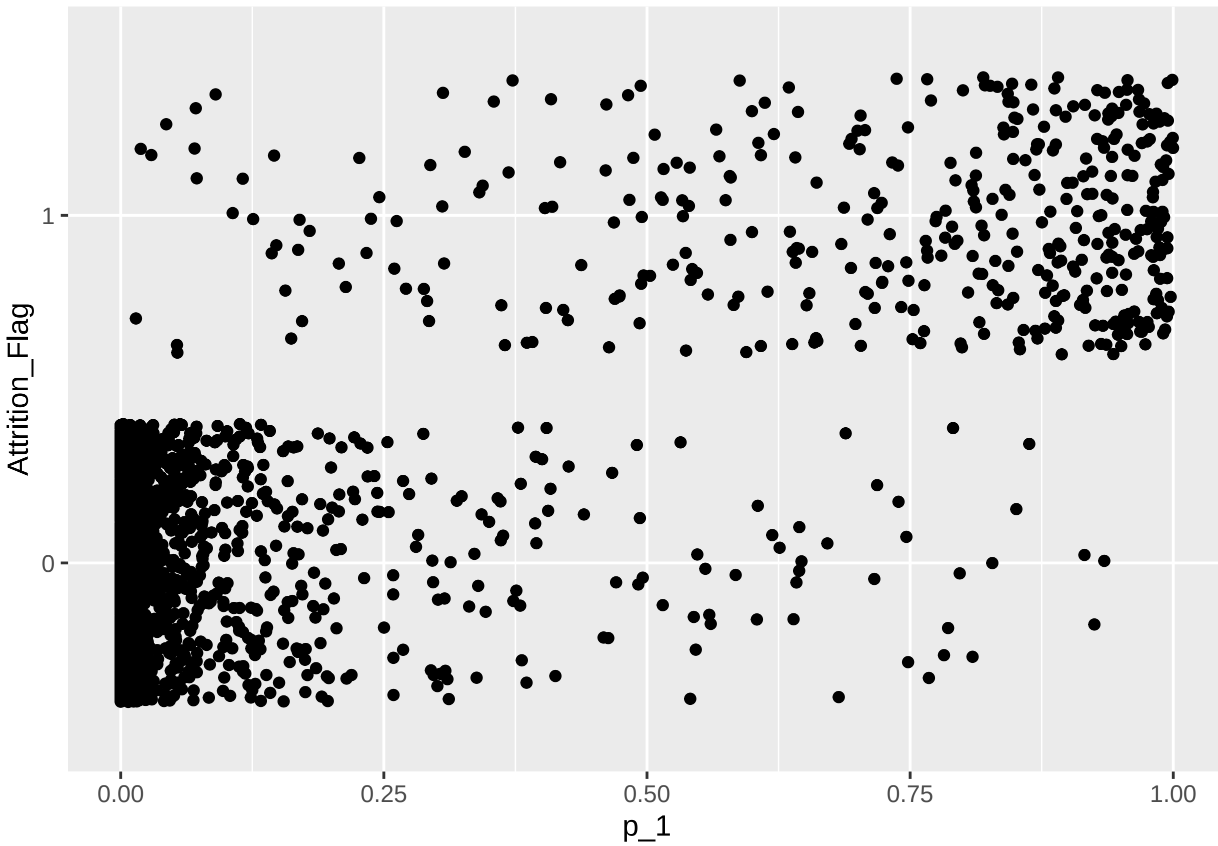

```
# compare to the null model
null_classification <- null_model() %>%
  set_engine("parsnip")%>%
  set_mode("classification")

null_rf <- workflow() %>%
  add_recipe(attrition_recipe_rf) %>%
  add_model(null_classification) %>%
  fit_resamples(
    cv_set
  )

null_metrics <- null_rf %>% collect_metrics()
```

```
################################### plot ROC curve ###################################
test_results <-
  test_set %>%
  select(Attrition_Flag) %>%
  bind_cols(
    last_rf_fit %>% collect_predictions() %>%
      select(p_1 = .pred_1)
  )

roc_data <- data.frame(threshold=seq(1,0,-0.01), fpr=0, tpr=0)
for (i in roc_data$threshold) {
```

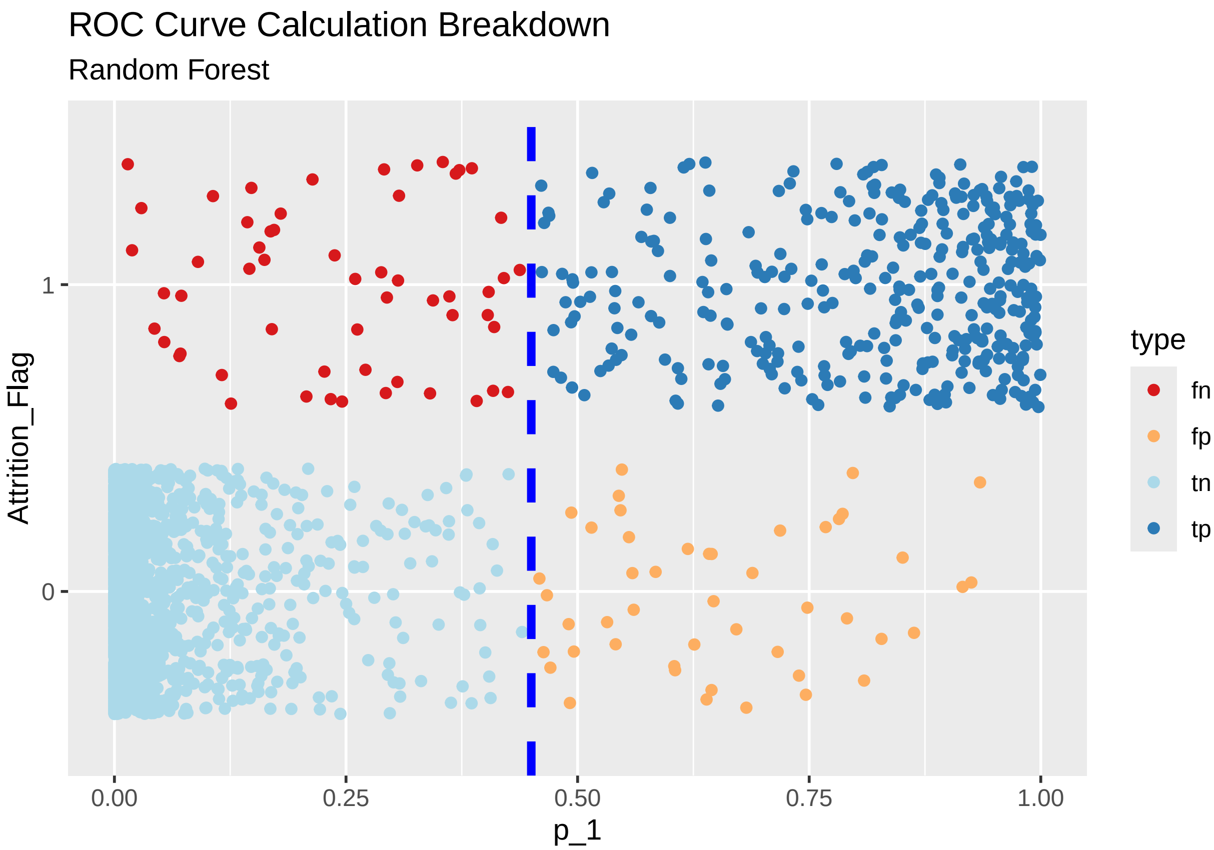
```r
  over_threshold <- test_results[test_results$p_1 >= i, ]

  fpr <- sum(over_threshold$Attrition_Flag==0)/sum(test_results$Attrition_Flag==0)
  roc_data[roc_data$threshold==i, "fpr"] <- fpr

  tpr <- sum(over_threshold$Attrition_Flag==1)/sum(test_results$Attrition_Flag==1)
  roc_data[roc_data$threshold==i, "tpr"] <- tpr

}

ggplot() +
  geom_line(data = roc_data, aes(x = fpr, y = tpr, color = threshold), linewidth = 2) +
  scale_color_gradientn(colors = rainbow(3)) +
  geom_abline(intercept = 0, slope = 1, lty = 2) +
  geom_point(data = roc_data[seq(1, 101, 10), ], aes(x = fpr, y = tpr)) +
  geom_text(data = roc_data[seq(1, 101, 10), ],
            aes(x = fpr, y = tpr, label = threshold, hjust = 1.2, vjust = -0.2)) +
  ggtitle("ROC Curve", subtitle = "Random Forest")
```




```r
############################## ROC curve calculation breakdown ############################

ggplot(data = test_results, aes(x = p_1, y = Attrition_Flag)) +
  geom_jitter()
```




```r
threshold <- 0.45

test_results$predictions <- ifelse(test_results$p_1 >= threshold, 1, 0)
tp <- nrow(test_results[test_results$Attrition_Flag==1 & test_results$predictions==1, ])
fp <- nrow(test_results[test_results$Attrition_Flag==0 & test_results$predictions==1, ])
tn <- nrow(test_results[test_results$Attrition_Flag==0 & test_results$predictions==0, ])
fn <- nrow(test_results[test_results$Attrition_Flag==1 & test_results$predictions==0, ])

test_results$type <- ""
test_results[test_results$Attrition_Flag==1 & test_results$predictions==1, "type"] <- "tp"
test_results[test_results$Attrition_Flag==0 & test_results$predictions==1, "type"] <- "fp"
test_results[test_results$Attrition_Flag==0 & test_results$predictions==0, "type"] <- "tn"
test_results[test_results$Attrition_Flag==1 & test_results$predictions==0, "type"] <- "fn"

ggplot(data = test_results, aes(x = p_1, y = Attrition_Flag)) +
  geom_jitter(aes(colour = type)) +
  geom_vline(xintercept = threshold, linetype = "dashed", color = "blue", linewidth = 1.5) +
  scale_color_brewer(palette = "RdYlBu") +
  ggtitle("ROC Curve Calculation Breakdown", subtitle = "Random Forest")
```




```
fpr <- fp/(fp + tn)
tpr <- tp/(tp + fn)

# confusion matrix data
tn
```

```
## [1] 2078
```

```
fn
```

```
## [1] 56
```

```
fp
```

```
## [1] 47
```

```
tp
```

```
## [1] 351
```